ConfigOps — Server Configuration Management Dashboard

Practical Assignment: CRUD Operations using Context API

Technology Stack

- React.js
- Context API
- useState
- React Router DOM
- Axios / Fetch API
- Local Backend Server
- Tailwind CSS / Bootstrap

Project Overview

Build a professional Server Configuration Management Dashboard where admins can manage:
- Development Servers
- Staging Servers
- Production Servers

The project must implement complete CRUD operations using Context API and local backend APIs.

Learning Objectives

- Understand Context API deeply
- Perform CRUD operations professionally
- Integrate frontend with local backend APIs
- Manage global state without Redux/useReducer
- Handle API loading and error states
- Understand GET, POST, PUT, PATCH, DELETE practically

Backend Configuration

Local Server URL:
http://localhost:5000

API Base Endpoint:
http://localhost:5000/api/servers

Required API Endpoints

```
GET     /api/servers
POST    /api/servers
PUT     /api/servers/:id
PATCH   /api/servers/:id
DELETE  /api/servers/:id
```

Expected Backend Data Structure

```
{
  "id": 1,
```

```
  "serverName": "Production-01",
  "ipAddress": "192.168.1.10",
  "environment": "Production",
  "version": "v2.4.1",
  "cpu": "8 Cores",
  "ram": "16GB",
  "status": "Running"
}
```

Folder Structure

```
src/
■
■■■ context/
■   ■■■ ServerContext.jsx
■
■■■ hooks/
■   ■■■ useServers.js
■
■■■ services/
■   ■■■ serverApi.js
■
■■■ components/
■   ■■■ ServerTable.jsx
■   ■■■ ServerForm.jsx
■   ■■■ SearchFilter.jsx
■   ■■■ StatusBadge.jsx
■   ■■■ DeleteModal.jsx
■
■■■ pages/
■   ■■■ Dashboard.jsx
■   ■■■ AddServer.jsx
■   ■■■ EditServer.jsx
■
■■■ routes/
■   ■■■ AppRoutes.jsx
■
■■■ App.jsx
```

Global Context State

```
const [servers, setServers] = useState([]);
const [loading, setLoading] = useState(false);
const [error, setError] = useState(null);
const [selectedServer, setSelectedServer] = useState(null);
```

Required Context Functions

```
fetchServers()
addServer()
updateServer()
patchServer()
deleteServer()
```

Module 1 — GET Operation

```
Requirements:
• Fetch all server configurations
• Store data globally using Context API
• Show loading state
• Handle API errors
• Display empty state if no data exists
```

Module 2 — POST Operation

```
Create a form with:
• Server Name
• IP Address
• Environment
• Version
• CPU Allocation
• RAM Allocation
• Status

Requirements:
• Validate all fields
• Prevent duplicate IPs
• Update UI instantly after creation
```

Module 3 — PUT Operation

```
Implement full update functionality.

Important:
Even if only one field changes, resend the entire object.
```

Module 4 — PATCH Operation

```
Implement quick actions:
• Stop Server
• Maintenance Mode
• Deploy New Version

PATCH should only send modified fields.
```

Module 5 — DELETE Operation

```
Requirements:
• Confirmation modal
• Instant UI update
• Rollback if delete fails
```

Module 6 — Search & Filtering

```
Implement:
• Search by server name
• Filter by environment
• Filter by status

Use useMemo() for optimization.
```

UI Requirements

```
• Responsive dashboard
• Sidebar layout
• Statistics cards
• Search panel
• Toast notifications
• Status badges
• Modal dialogs
• Empty state screens
```

Advanced Challenges

1. Auto refresh every 30 seconds
2. Activity timeline tracking
3. Bulk actions
4. Dark mode with localStorage

Evaluation Criteria

- Context API Usage — 20 Marks
- CRUD Functionality — 30 Marks
- API Integration — 15 Marks
- UI/UX — 10 Marks
- Error Handling — 10 Marks
- Optimization — 10 Marks
- Code Structure — 5 Marks

Deliverables

- GitHub Repository
- Working Frontend
- Connected Local Backend
- README.md
- Screenshots
- API Documentation

Final Outcome

Students will be able to:
- Build scalable React dashboards
- Implement professional CRUD systems
- Connect frontend with local backend servers
- Manage global state using Context API
- Structure enterprise-level frontend applications